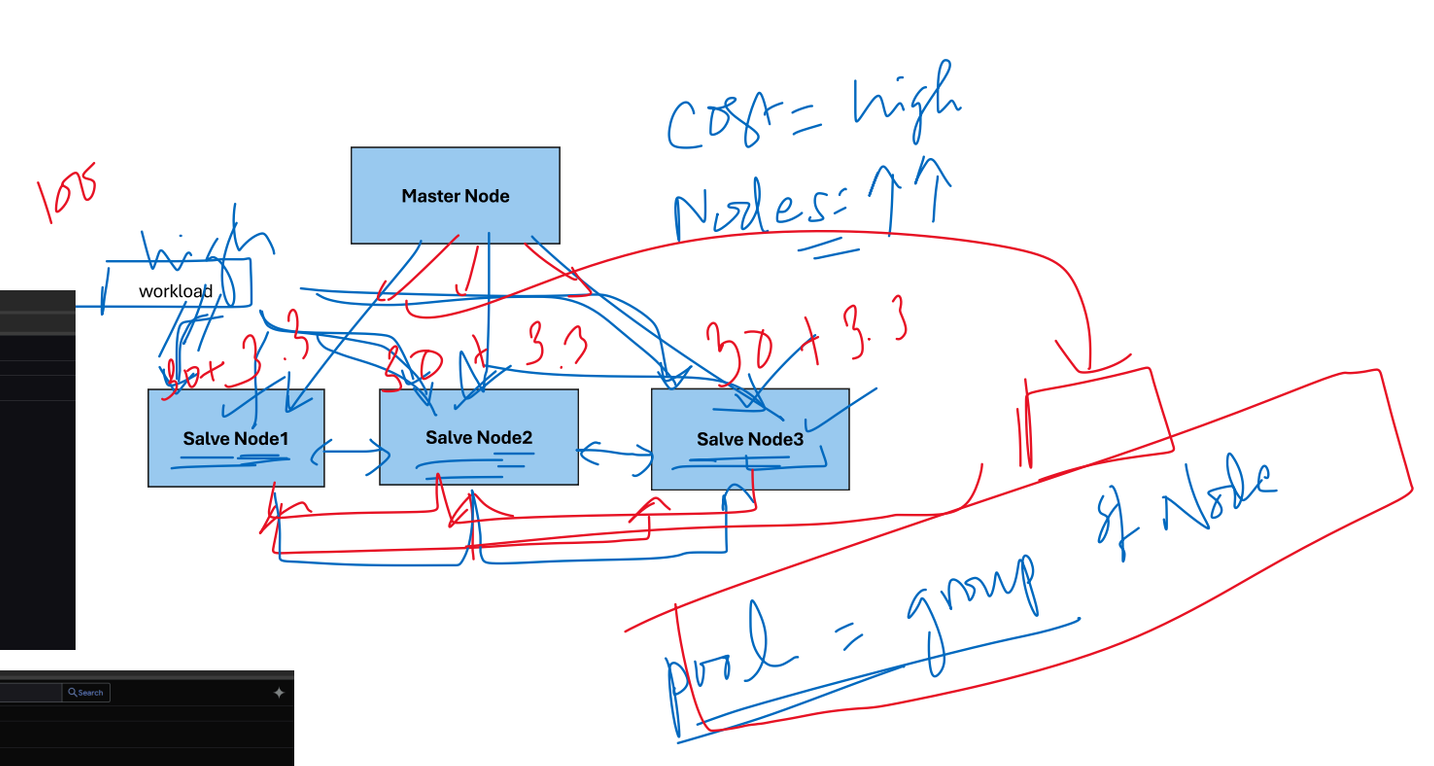
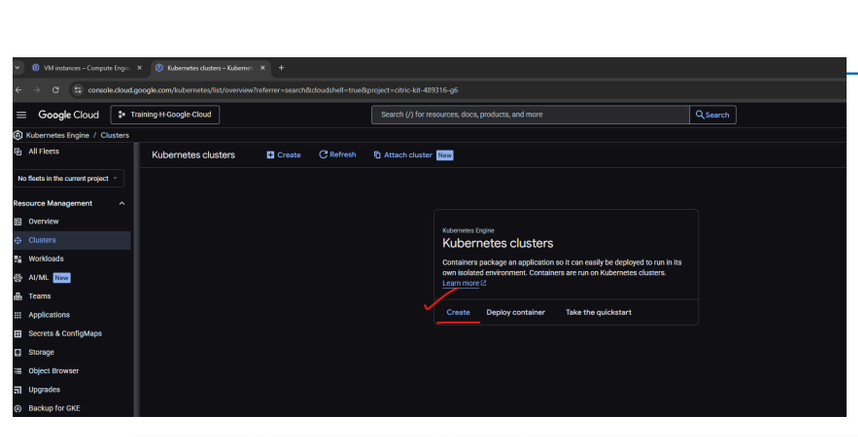
DAY5 GKE Kubernetes - Observability and Operational Management

Standard - Mode



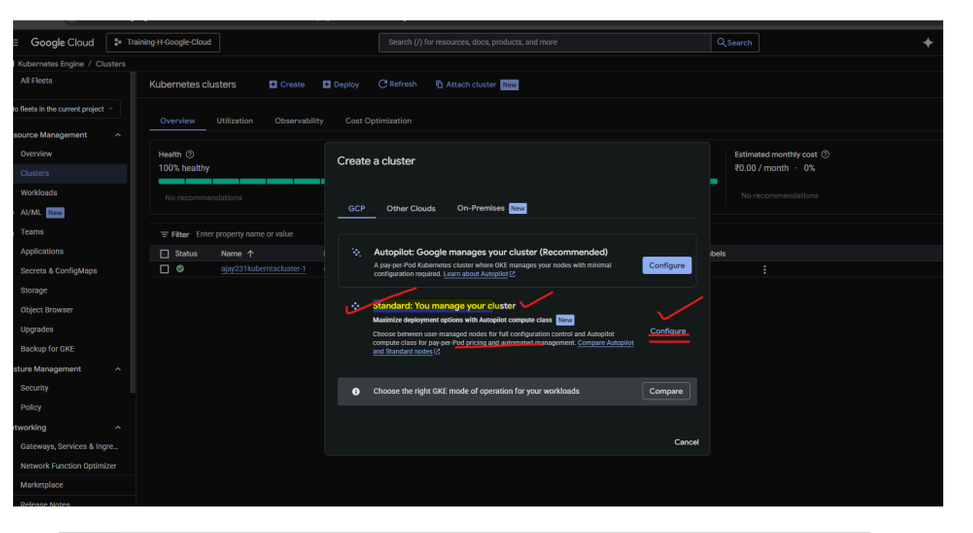
100
high
workload
Master Node
cost = high
Nodes: ↑↑
30+3.3
30+3.3
30+3.3
Salve Node1
Salve Node2
Salve Node3
pool = group of Node



Kubernetes clusters
Create
Deploy container
Take the quickstart

Kubernetes clusters

Containers package an application so it can easily be deployed to run in its own isolated environment. Containers are run on Kubernetes clusters.

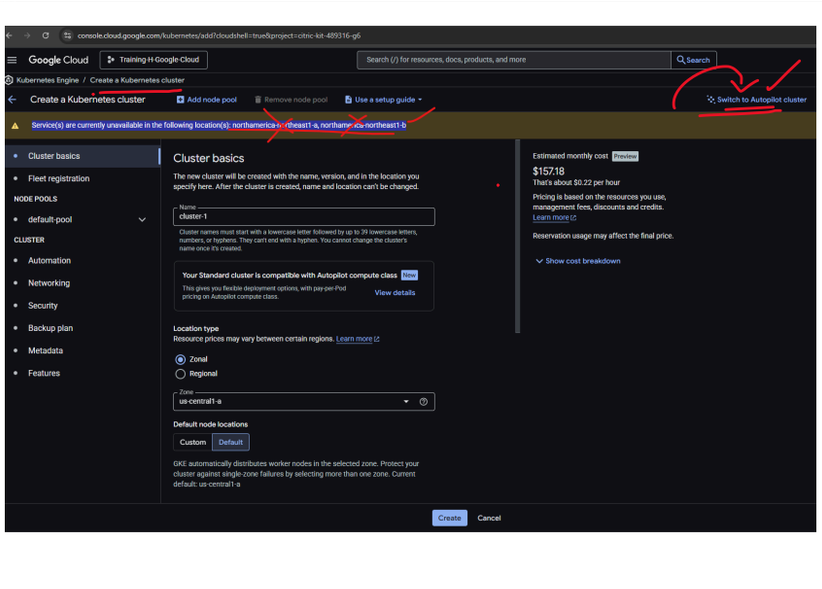


Create a cluster
GCP
Other Clouds
On-Premises
Configure
Compare
Cancel

Autopilot: Google manages your cluster (Recommended)

Standard: You manage your cluster

Choose the right GKE mode of operation for your workloads



Create a Kubernetes cluster
Switch to Autopilot cluster
Cluster basics
Fleet registration
NODE POOLS
default-pool
CLUSTER
Automation
Networking
Security
Backup plan
Metadata
Features
Create
Cancel

Cluster basics

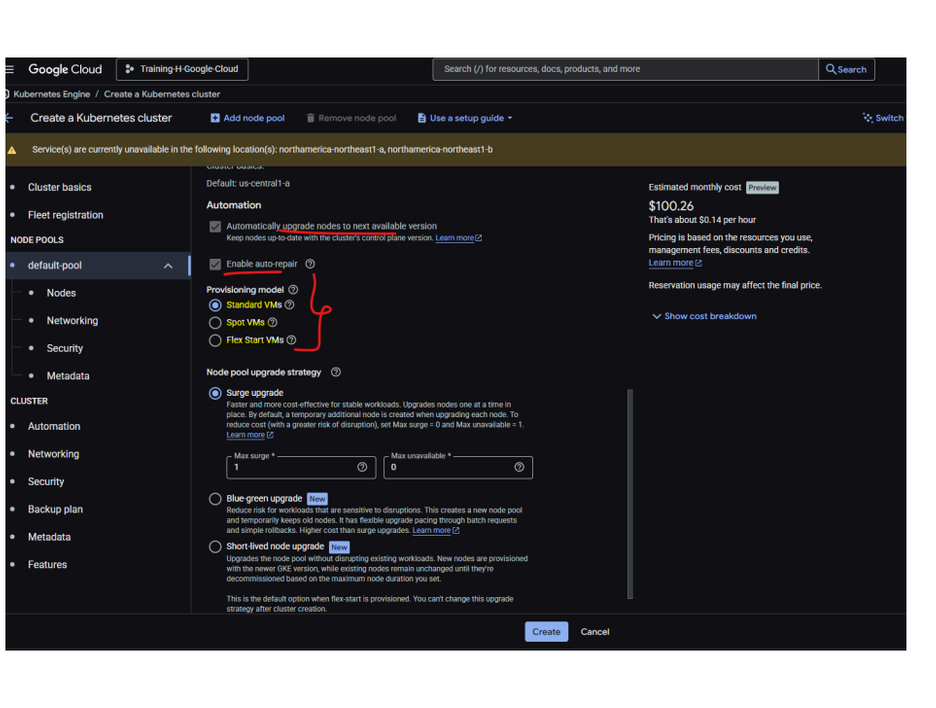
The new cluster will be created with the name, version, and in the location you specify here. After the cluster is created, name and location can't be changed.

Name: cluster-1

Location type: ● Zonal ○ Regional

Zone: us-central1-a

Estimated monthly cost: $157.18



Create a Kubernetes cluster
Create
Cancel

Automation

☑ Automatically upgrade nodes to next available version

☑ Enable auto-repair

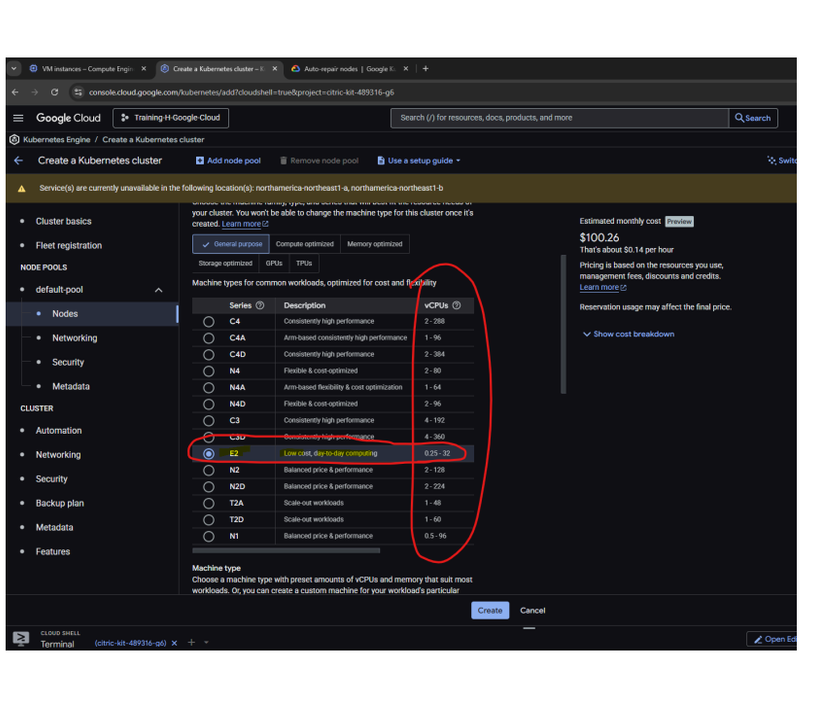
Provisioning model: ● Standard VMs ○ Spot VMs ○ Flex Start VMs

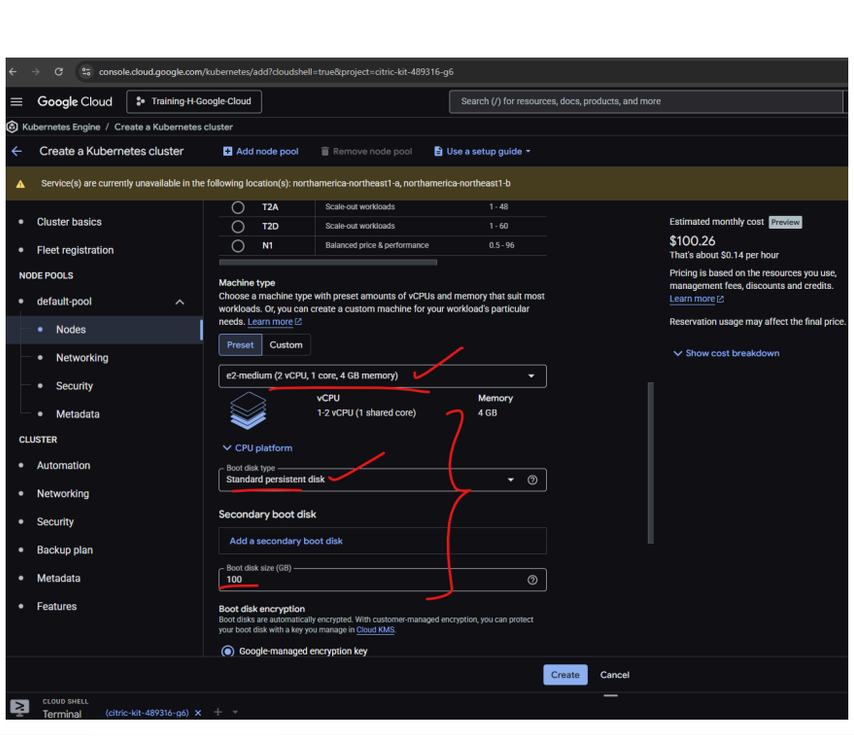
Node pool upgrade strategy: ● Surge upgrade ○ Blue-green upgrade ○ Short-lived node upgrade

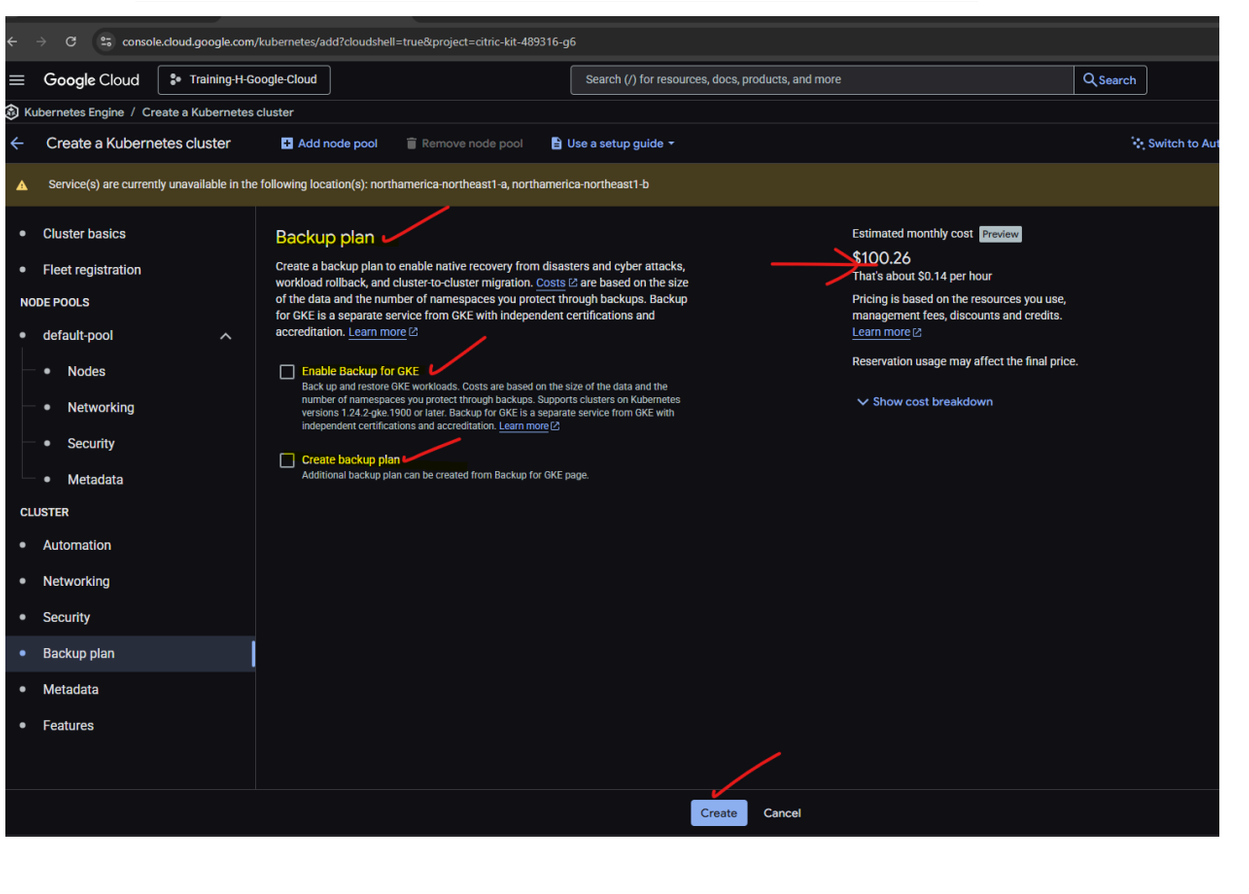
Max surge: 1

Max unavailable: 0

Estimated monthly cost: $100.26







Backup plan
Create
Cancel

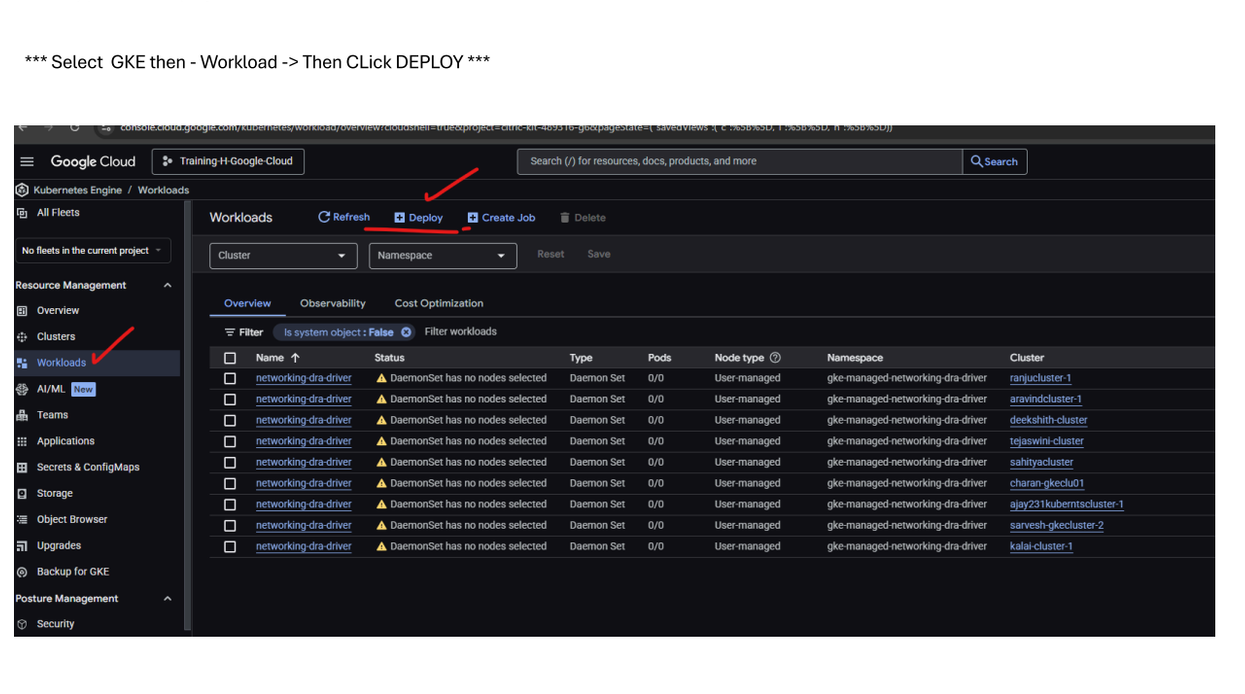
Fleet: A Fleet (Group) in GKE is a logical grouping of multiple Kubernetes clusters (GKE or external) under a single management layer.

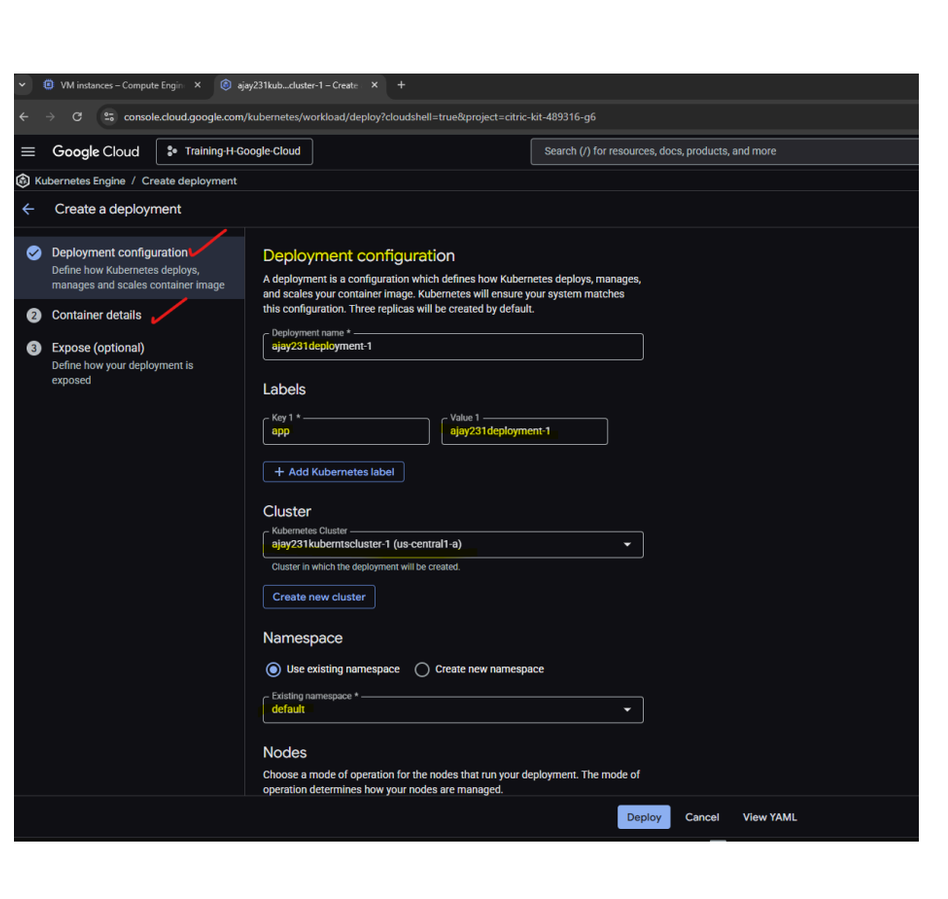
It enables:

- Centralized management
- Unified security policies
- Cross-cluster service discovery
- Multi-cluster observability

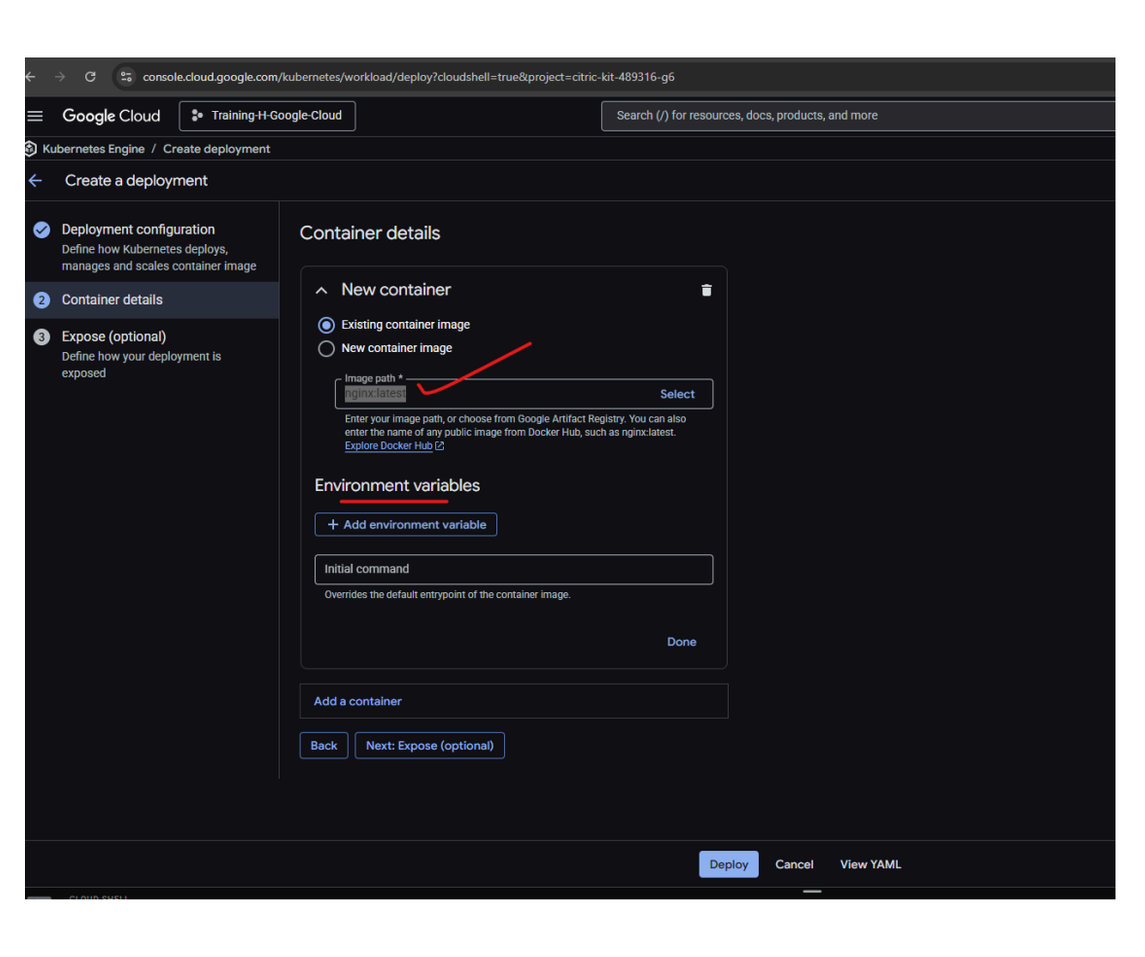
GKE - Application deployment demo :

*** Select GKE Item - Workload -> Then CLick DEPLOY ***

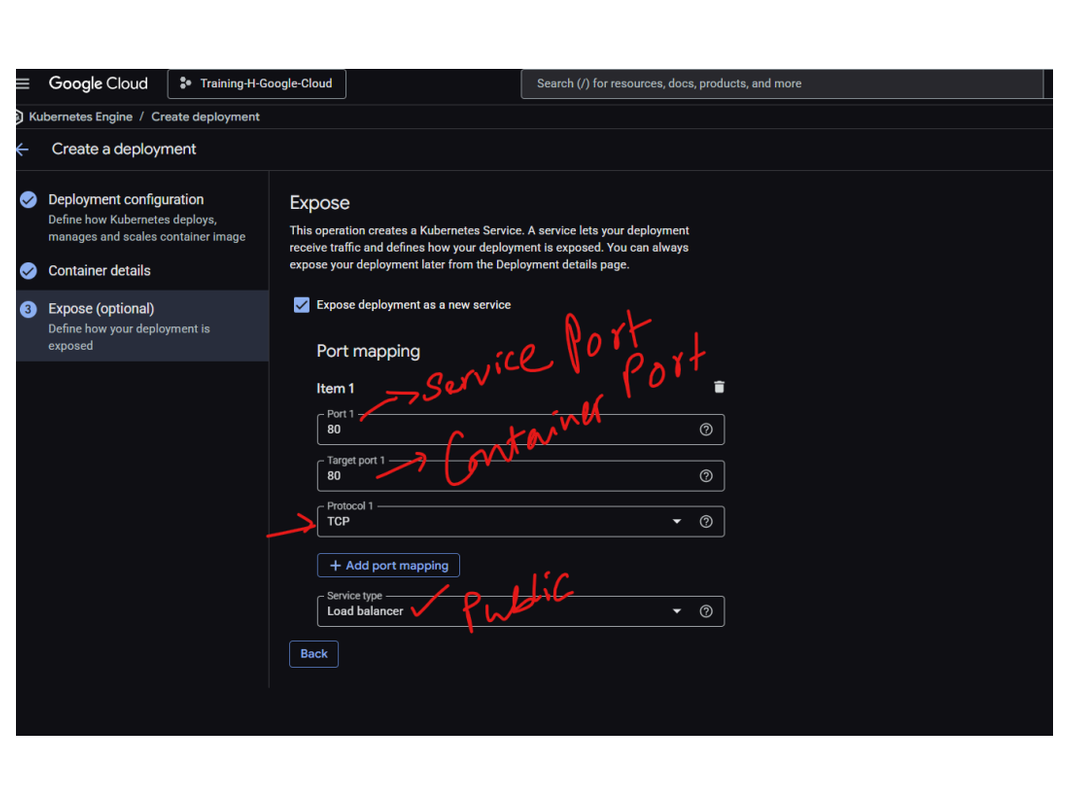




Create a deployment
Deployment configuration
Container details
Expose (optional)



Container details
New container
Existing container image
New container image
Environment variables
Add environment variable
Initial command
Done
Add a container
Back
Next: Expose (optional)
Deploy
Cancel
View YAML



Create a deployment
Deployment configuration
Container details
Expose (optional)
Add port mapping
Back
Service Port
Container Port
Public

Expose

This operation creates a Kubernetes Service. A service lets your deployment receive traffic and defines how your deployment is exposed. You can always expose your deployment later from the Deployment details page.

☑ Expose deployment as a new service

Port mapping

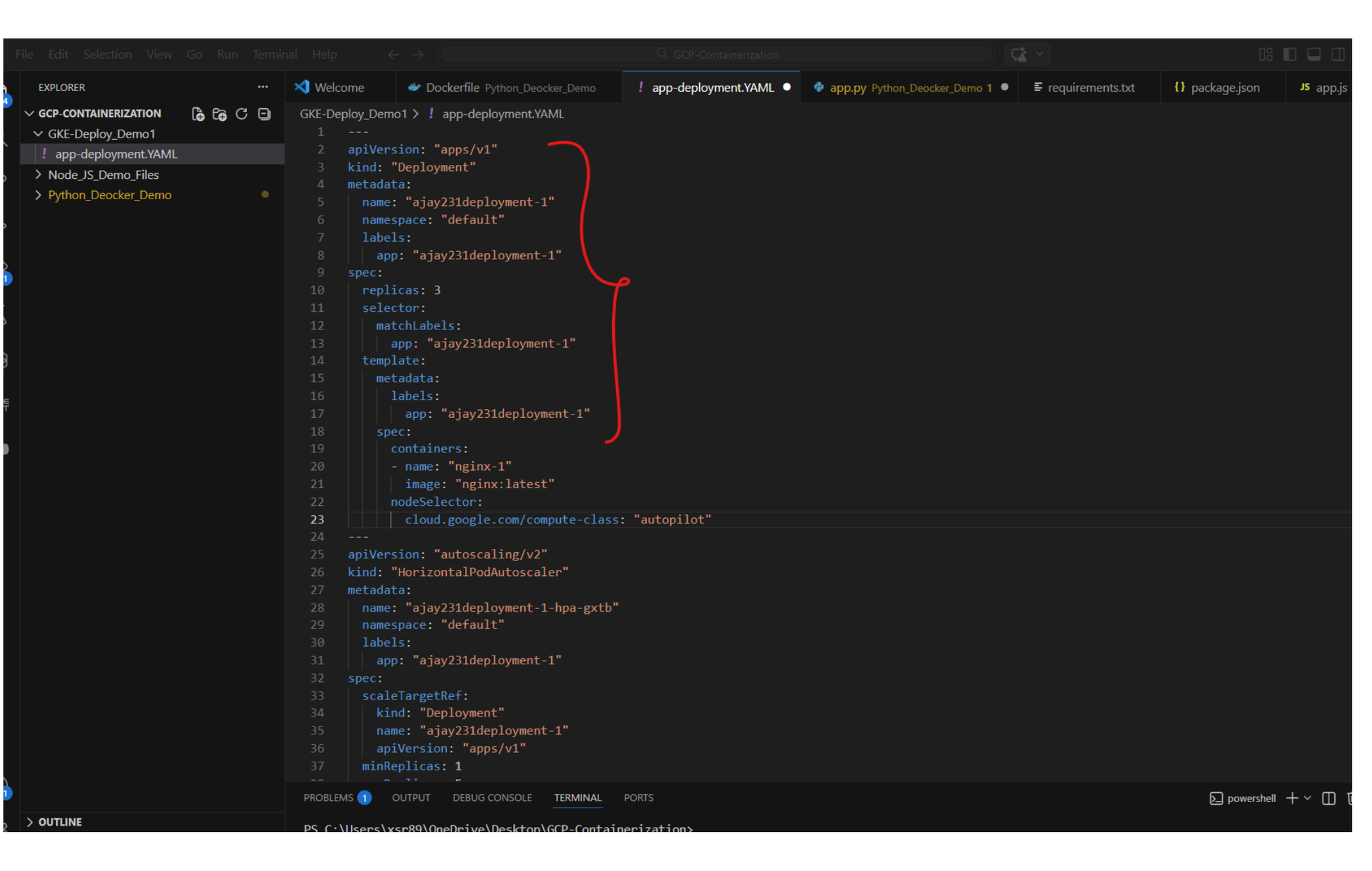
Item 1

Port 1: 80

Target port 1: 80

Protocol 1: TCP

Service type: Load balancer



```yaml
---
apiVersion: "apps/v1"
kind: "Deployment"
metadata:
  name: "ajay231deployment-1"
  namespace: "default"
  labels:
    app: "ajay231deployment-1"
spec:
  replicas: 3
  selector:
    matchLabels:
      app: "ajay231deployment-1"
  template:
    metadata:
      labels:
        app: "ajay231deployment-1"
    spec:
      containers:
      - name: "nginx-1"
        image: "nginx:latest"
      nodeSelector:
        cloud.google.com/compute-class: "autopilot"
---
apiVersion: "autoscaling/v2"
kind: "HorizontalPodAutoscaler"
metadata:
  name: "ajay231deployment-1-hpa-gxtb"
  namespace: "default"
  labels:
    app: "ajay231deployment-1"
spec:
  scaleTargetRef:
    kind: "Deployment"
    name: "ajay231deployment-1"
    apiVersion: "apps/v1"
  minReplicas: 1
```

** Horizontal Pod Autoscaler (HPA):

Automatically scales Pods (replicas) based on metrics like:

- CPU utilization (most common) [If CPU > 70% → scale from 2 → 5 Pods] [If CPU drops → scale back down]
- Memory usage
- Custom metrics (advanced)

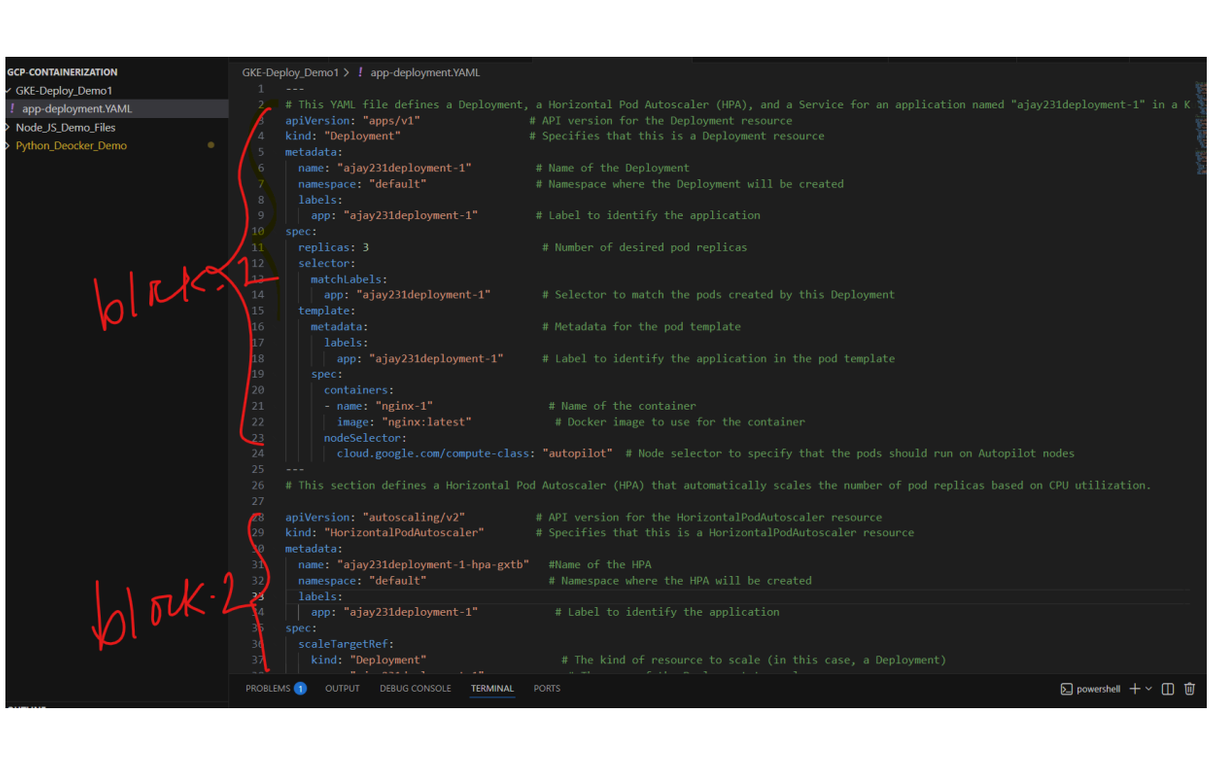
** How HPA Works
- Reads metrics from Metrics Server
- Compares with target (e.g., 70% CPU)
- Adjusts Deployment replicas automatically

*Expected Behavior

- CPU increases - HPA triggers scaling: --> 2 → 4 → 6 Pods (depending on load)

- When load stops → scales down

YAML - FILE configs details:

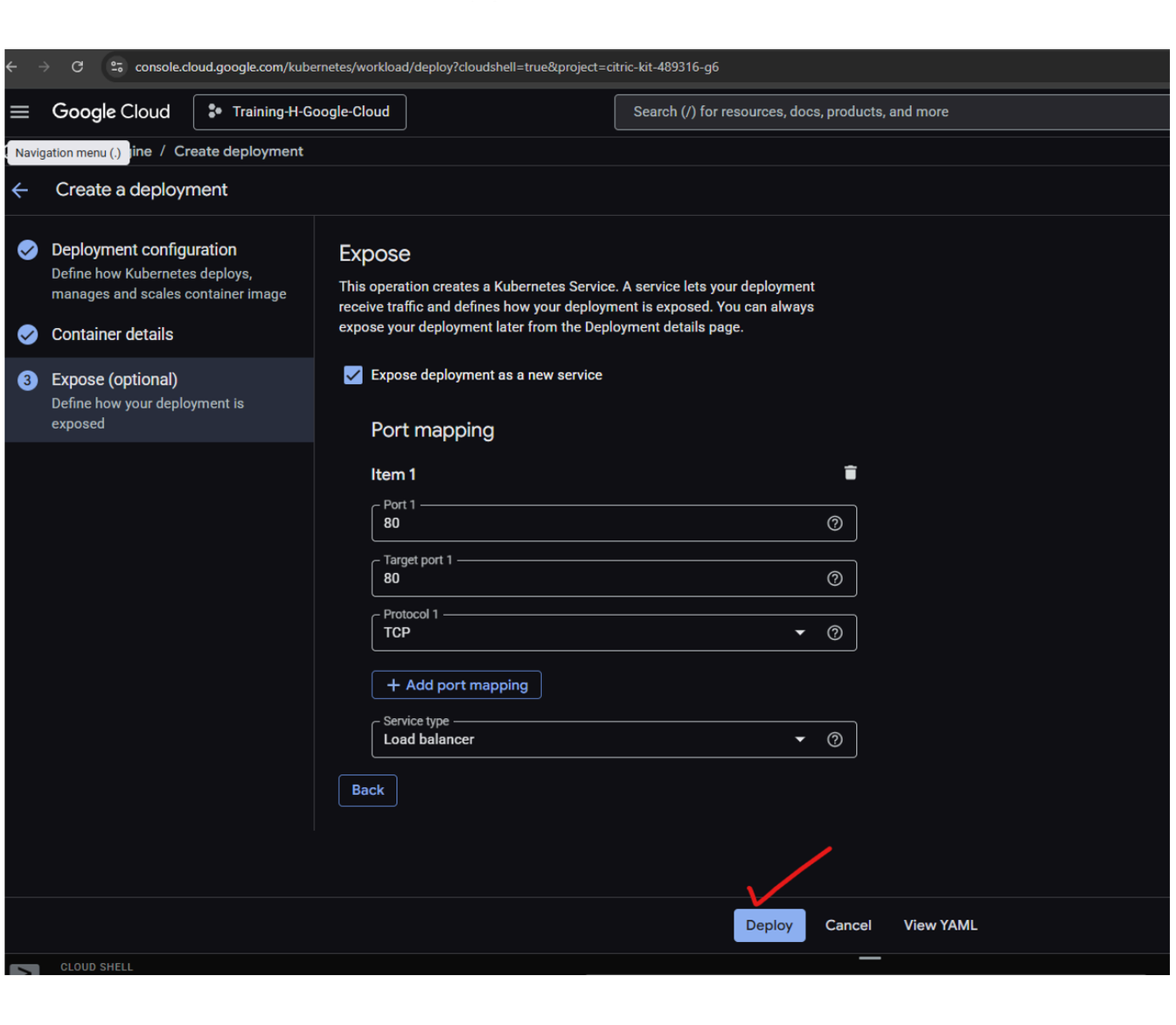


block1
block 2

```yaml
---
# This YAML file defines a Deployment, a Horizontal Pod Autoscaler (HPA), and a Service for an application named "ajay231deployment-1" in a K
apiVersion: "apps/v1"                    # API version for the Deployment resource
kind: "Deployment"                       # Specifies that this is a Deployment resource
metadata:
  name: "ajay231deployment-1"            # Name of the Deployment
  namespace: "default"                   # Namespace where the Deployment will be created
  labels:
    app: "ajay231deployment-1"           # Label to identify the application
spec:
  replicas: 3                            # Number of desired pod replicas
  selector:
    matchLabels:
      app: "ajay231deployment-1"         # Selector to match the pods created by this Deployment
  template:
    metadata:                            # Metadata for the pod template
      labels:
        app: "ajay231deployment-1"       # Label to identify the application in the pod template
    spec:
      containers:
      - name: "nginx-1"                  # Name of the container
        image: "nginx:latest"            # Docker image to use for the container
      nodeSelector:
        cloud.google.com/compute-class: "autopilot"  # Node selector to specify that the pods should run on Autopilot nodes
---
# This section defines a Horizontal Pod Autoscaler (HPA) that automatically scales the number of pod replicas based on CPU utilization.

apiVersion: "autoscaling/v2"             # API version for the HorizontalPodAutoscaler resource
kind: "HorizontalPodAutoscaler"          # Specifies that this is a HorizontalPodAutoscaler resource
metadata:
  name: "ajay231deployment-1-hpa-gxtb"   #Name of the HPA
  namespace: "default"                   # Namespace where the HPA will be created
  labels:
    app: "ajay231deployment-1"           # Label to identify the application
spec:
  scaleTargetRef:
    kind: "Deployment"                   # The kind of resource to scale (in this case, a Deployment)
```

After the YAML - Details cross Verification - Please click on Deploy



Create a deployment
Deployment configuration
Container details
Expose (optional)
Add port mapping
Back
Deploy
Cancel
View YAML

Expose

This operation creates a Kubernetes Service. A service lets your deployment receive traffic and defines how your deployment is exposed. You can always expose your deployment later from the Deployment details page.

☑ Expose deployment as a new service

Port mapping

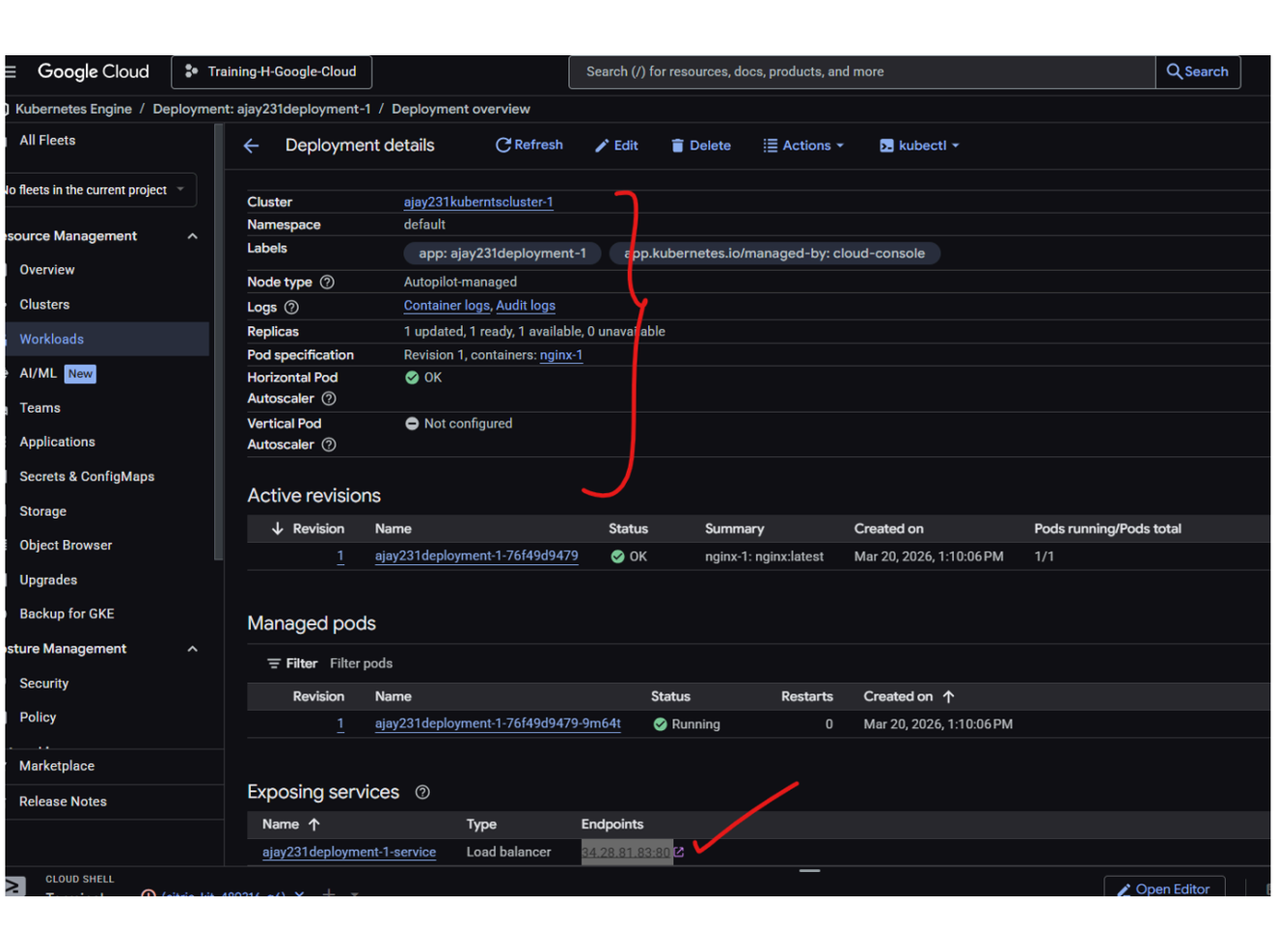
Item 1

Port 1: 80

Target port 1: 80

Protocol 1: TCP

Service type: Load balancer



Deployment details
Refresh
Edit
Delete
Actions
kubectl
Active revisions
Managed pods
Exposing services

Cluster: ajay231kubernetscluster-1

Namespace: default

Labels: app: ajay231deployment-1 | app.kubernetes.io/managed-by: cloud-console

Node type: Autopilot-managed

Logs: Container logs, Audit logs

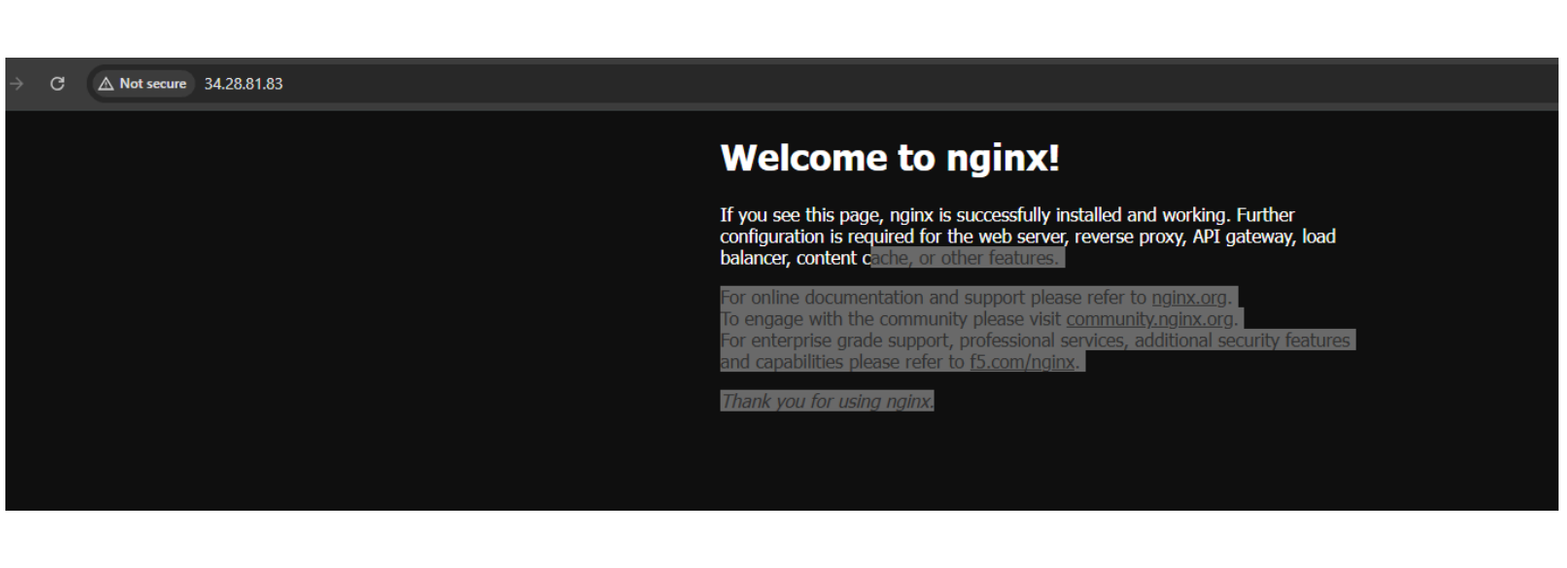
Replicas: 1 updated, 1 ready, 1 available, 0 unavailable

Pod specification: Revision 1, containers: nginx-1

Horizontal Pod Autoscaler: OK

Vertical Pod Autoscaler: Not configured

*** Go to the Deployment then --> Endpoints -->34.28.81.83:80 CLick on it & you see Message in WEBPAGE



Not secure 34.28.81.83

Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

Cloud Monitoring & Logging in GCP:

Cloud Monitoring → Collects and visualizes metrics
(CPU, memory, latency, uptime)

Cloud Logging → Collects and stores logs
(application logs, system logs, error logs)

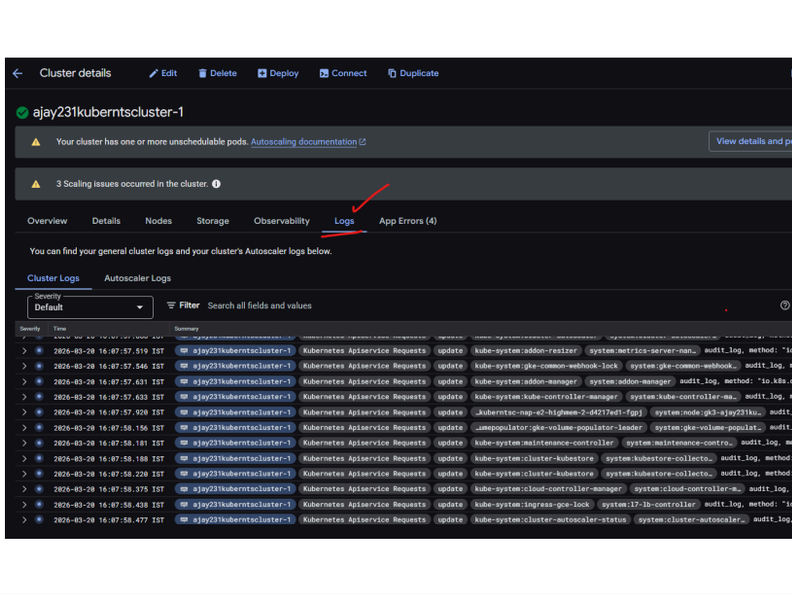
In Kubernetes:

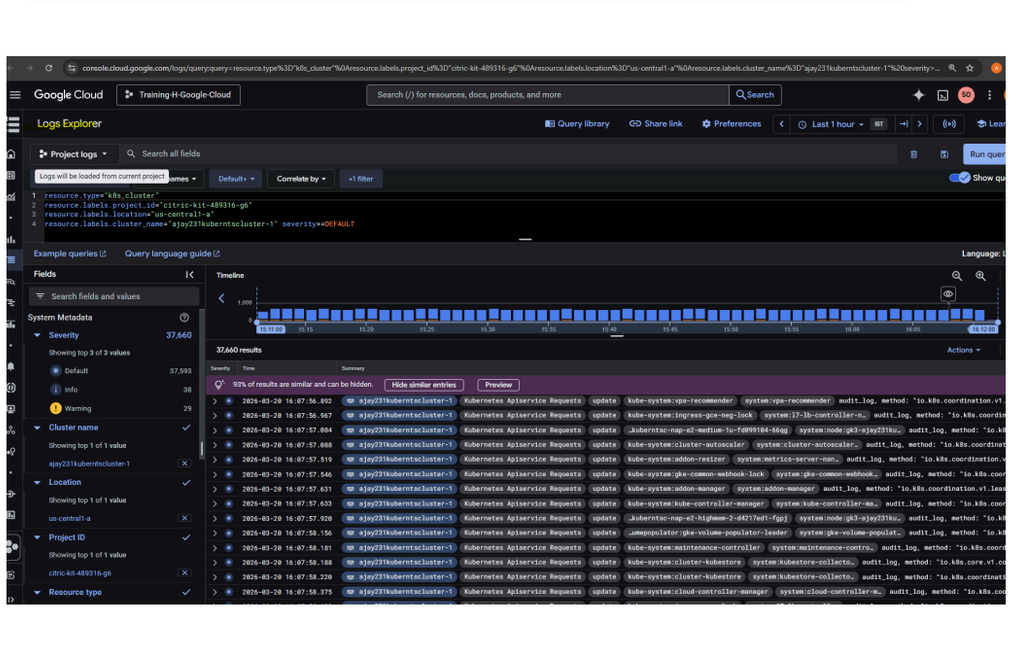
Pods → generate logs + metrics
↓
GCP collects them automatically
↓
Used for monitoring + troubleshooting

Dashboards = Visual representation of system health

Log-based metrics = Convert logs into measurable metrics

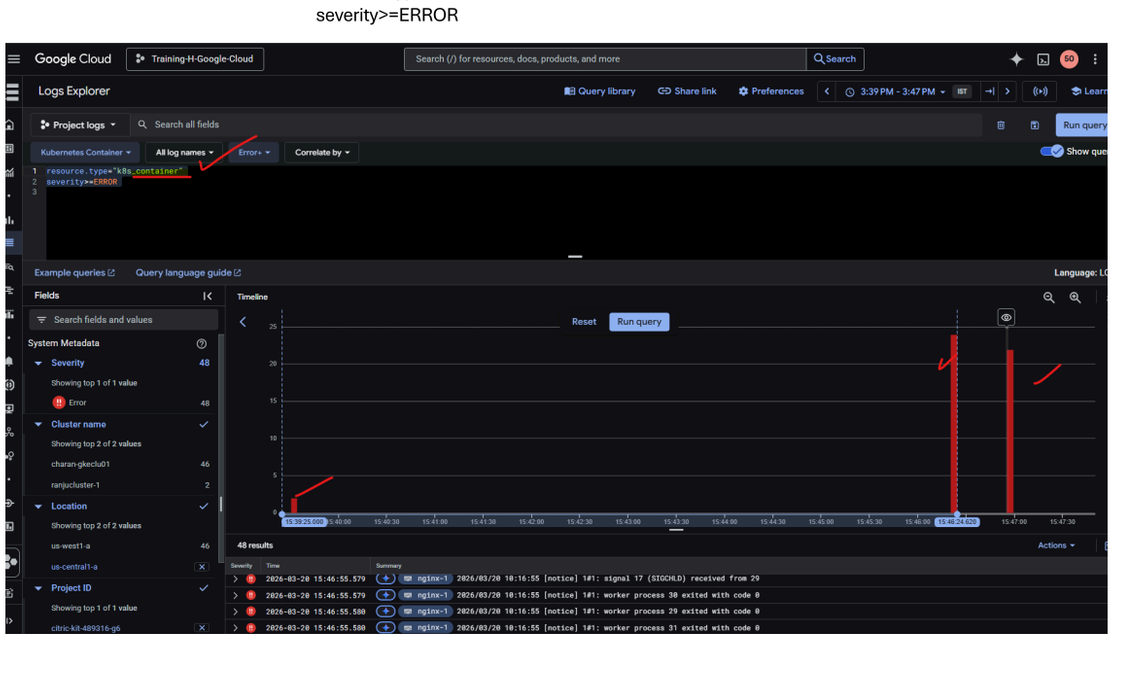
** Demo: Create Log alerts in GKE - K8S

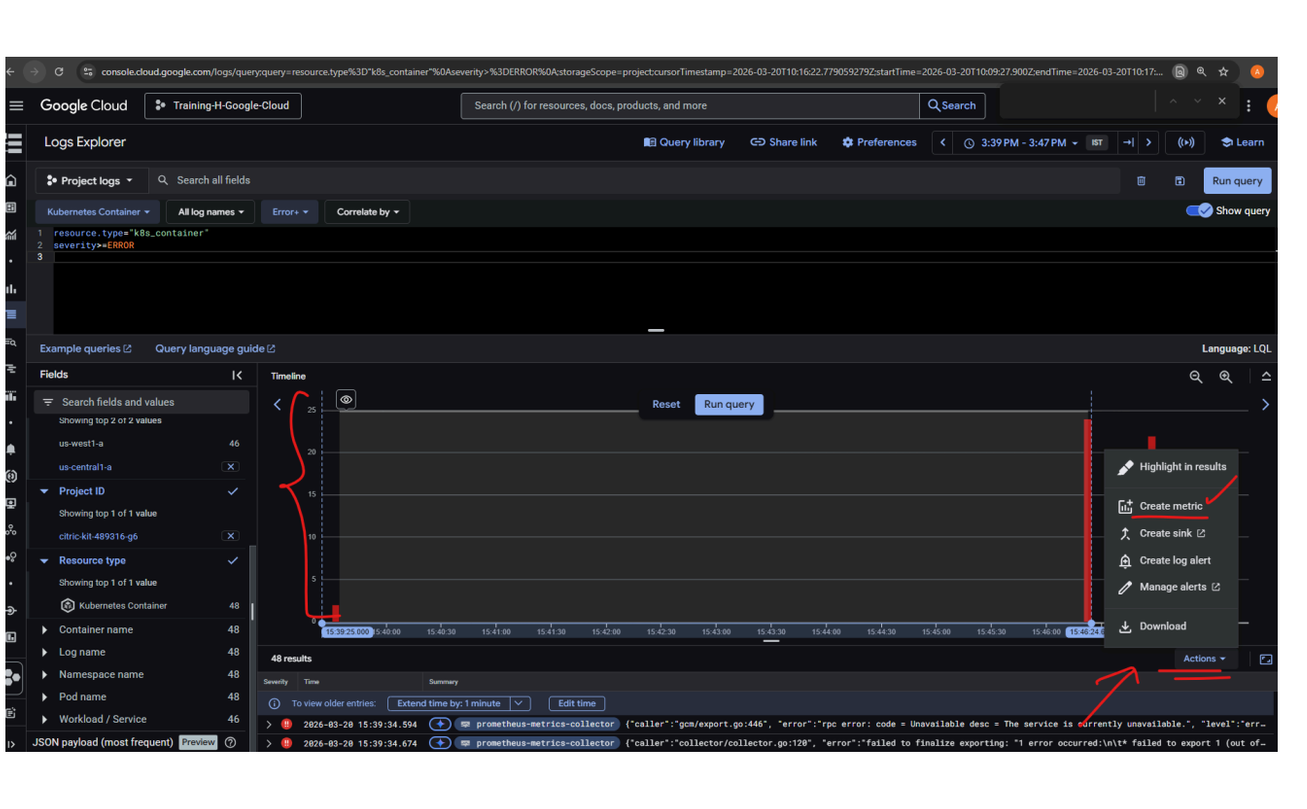


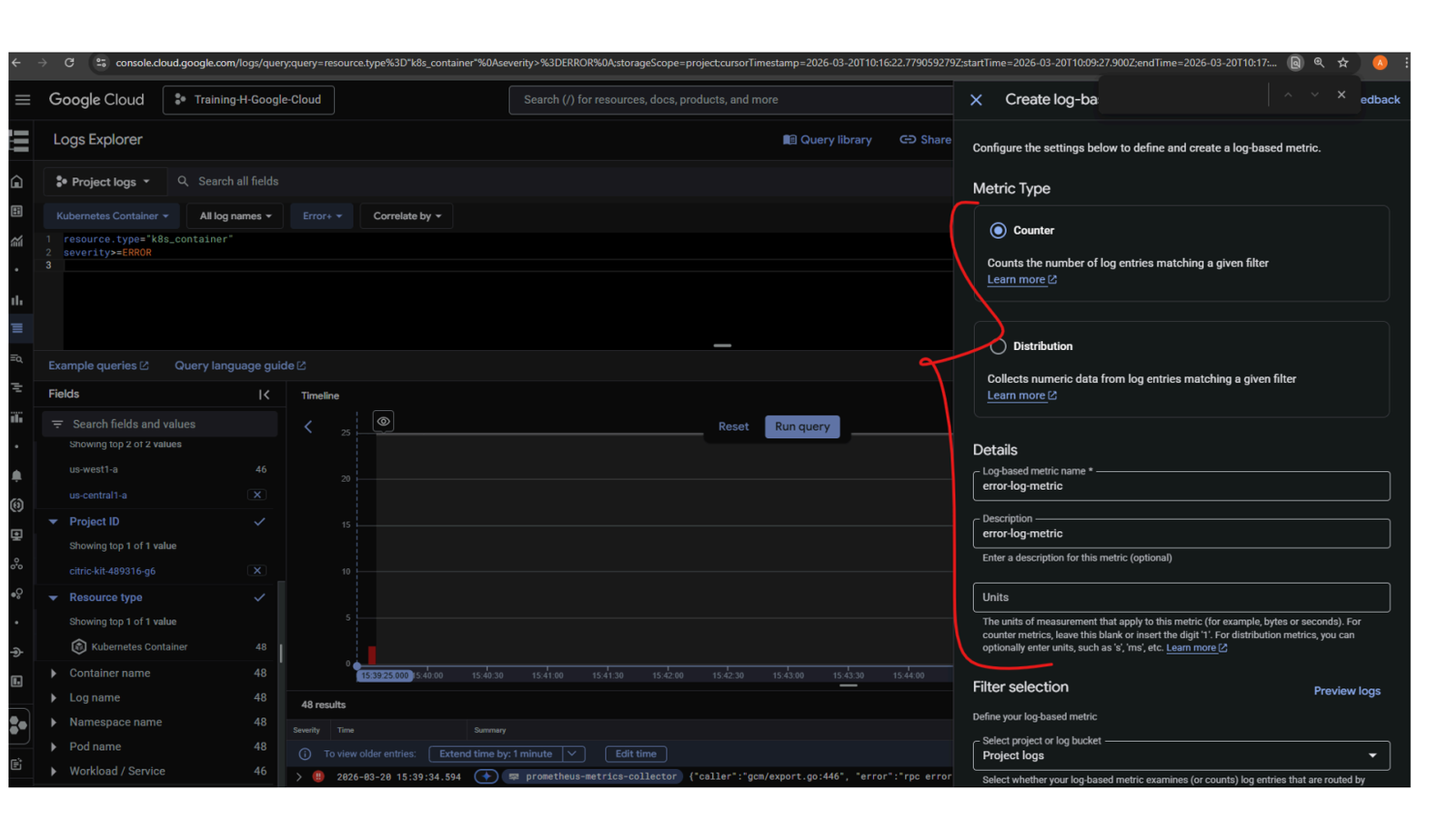


SAMPLE QUERY to check Errors

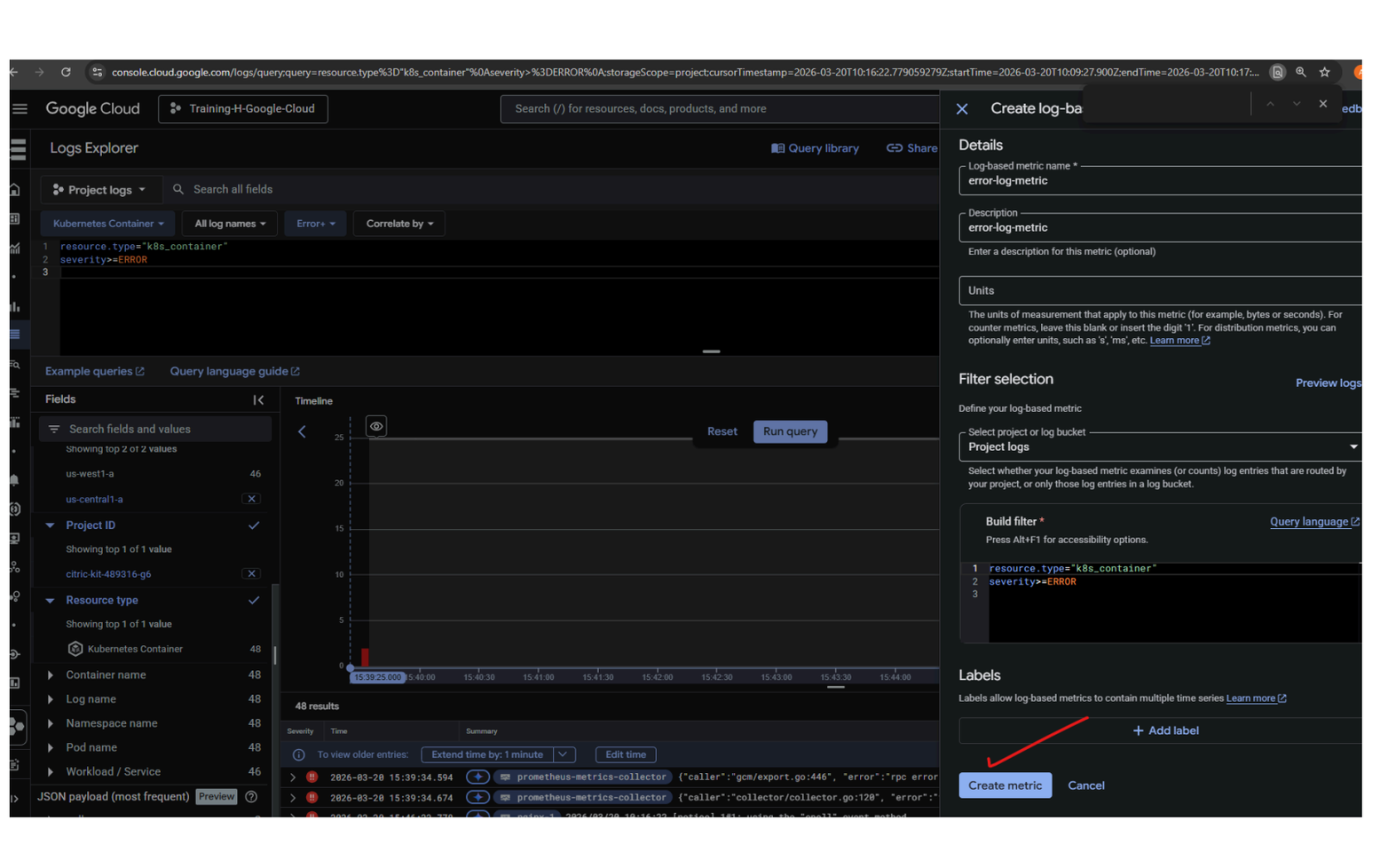
resource.type="k8s_container"
severity>=ERROR





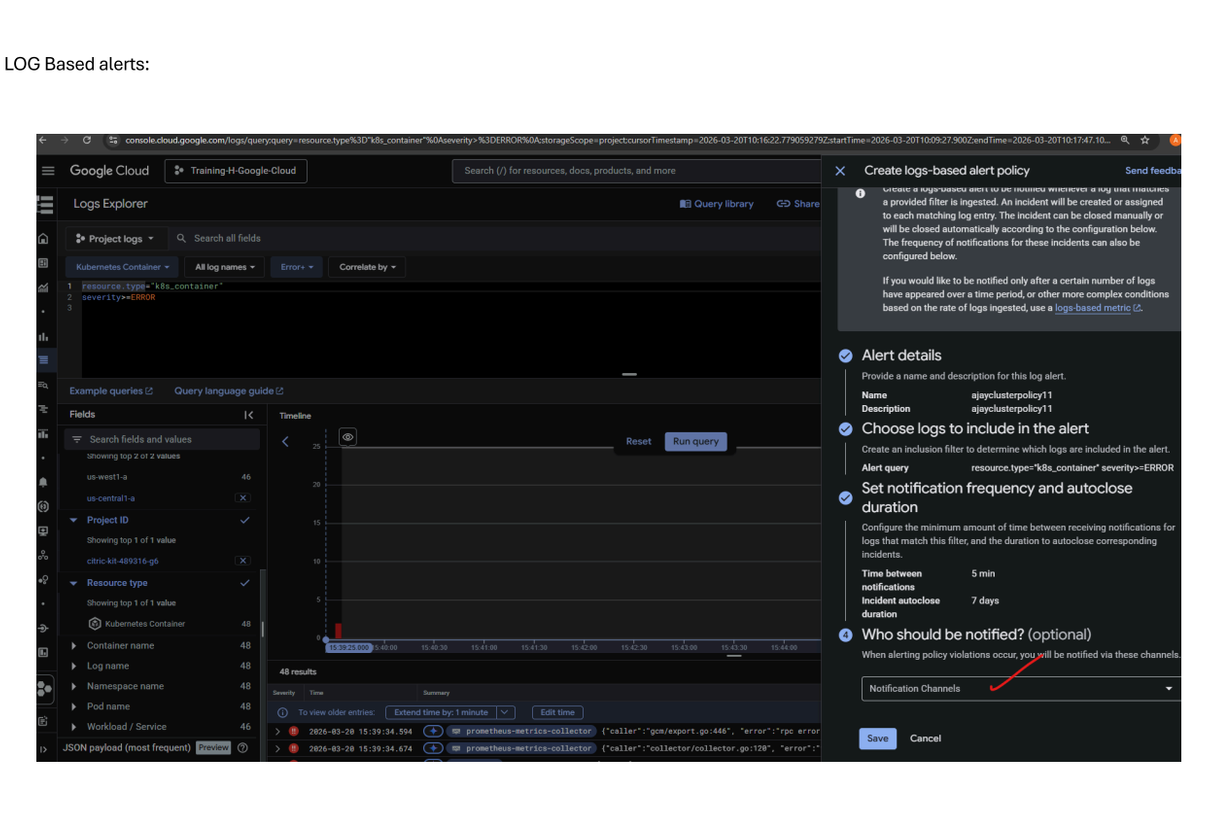


Create log-ba
Metric Type
Counter
Distribution
Details
Filter selection



Create log-ba
Details
Filter selection
Labels
Add label
Create metric
Cancel

LOG Based alerts:



Set notification frequency and autoclose duration
Who should be notified? (optional)
Notification Channels
Save
Cancel

Mini Project 1: E-Commerce Website Monitoring (Flipkart/Amazon Style)

Use Case: Monitor an online shopping website to ensure it is always available and performant.:

What You'll Build

- Deploy web app (nginx as website)
- Monitor CPU, memory
- Track logs

STEP1 -> Deploy Application - write app code & Create Application image -> put the image into Artifact registry

STEP2: Pull it into -> GKE - CLuster - Deployment -> Expose Application , PORT : 80, & CONTAINER PORT: 80

STEP3: Access Website -> ENDPOINTS -> Copy External IP → Open in browser

STEP4: Go to Monitoring → Dashboards → GKE Workloads
Show: CPU usage , Memory usage , Pod count

Step 5: View Logs -> Logging → Logs Explorer

QUERY option & Create LOG Alerts !!

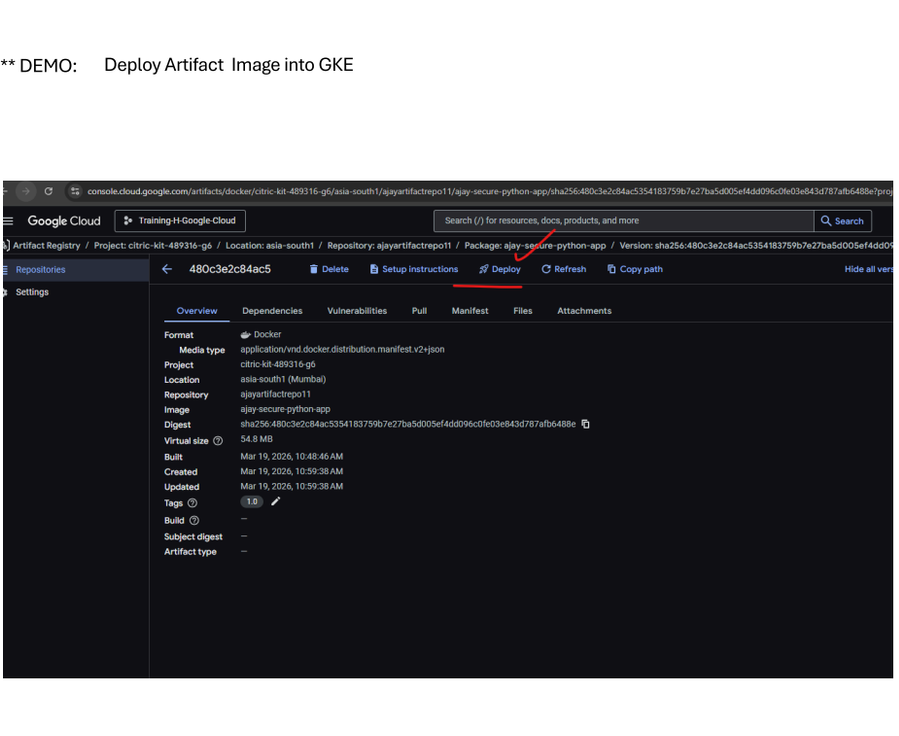
✔ Website deployed
✔ Live monitoring
✔ Logs visible

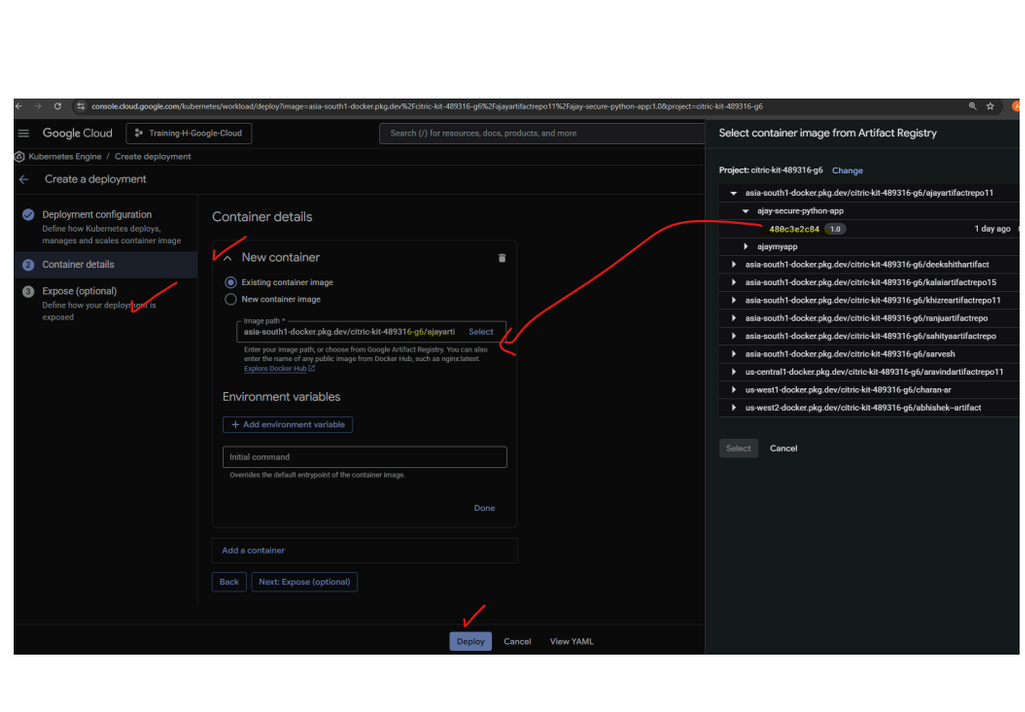
** Mini Project 2: Auto-Scaling for Food Delivery App (Swiggy/Zomato)

* Use Case: Scale application automatically during peak hours (lunch/dinner rush).

* What You'll Build:
- Enable auto-scaling - to handle Higher workloads during PEAK Hours/ TIME
- Handle high traffic -

*** DEMO: Deploy Artifact Image into GKE





Create a deployment
Container details
New container
Environment variables
Add a container
Select container image from Artifact Registry
Select
Cancel
Deploy
Cancel
View YAML